תרגיל 0 – סביבת עבודה והתקנות

בתרגיל זה נכיר את סביבת העבודה שבה נשתמש לאורך הקורס, נתקין את הכלים הדרושים לפיתוח תוכניות מחשב בשפת C ונתנסה לראשונה בעבודה איתם.
בסיום התרגיל תוכלו לכתוב ולערוך קוד בשפת C, לקמפל ולהריץ תוכניות במחשב האישי שלכם, לנהל גרסאות של הקוד באמצעות Git ולהגיש תרגילים במערכת ההגשה של הקורס.
שימו לב: תרגיל זה נועד להיכרות ראשונית עם סביבת העבודה ואינו נכלל בחישוב הציון הסופי בקורס. אין צורך בניסיון קודם עם הכלים המוצגים בו מעבר לנלמד בתרגול.
לאחר השלמת כל שלבי התרגיל והגשתו בהצלחה, תקבלו הודעת דוא"ל אוטומטית ממערכת ההגשה ובה ציון 100.
בתרגיל זה, הציון מעיד על כך שההגשה בוצעה בהצלחה ושהבדיקות הנדרשות עברו.

לפני שמתחילים

לפני שמתחילים עליכם לוודא שיש ברשותכם את שם המשתמש והסיסמה המחלקתיים.
אתם יכולים למצוא את שם המשתמש כאן, תחת הערך UNIX-MACS או ESC.
הסיסמה היא הסיסמה המשמשת אתכם להתחברות למערכת אינבר.

במהלך הקורס נשתמש ב-GitHub לצורך עבודה עם מאגרי הקוד של התרגילים ולמעקב אחר השינויים שתבצעו בקוד. לשם כך, נדרש חשבון GitHub אישי.
אם עדיין אין ברשותכם חשבון, היכנסו לאתר GitHub וצרו חשבון ללא עלות.

מומלץ ליצור במחשב תיקייה ייעודית עבור התרגילים בקורס, ובתוכה תיקייה נפרדת לכל תרגיל.
מומלץ להשתמש בשמות תיקיות באנגלית בלבד, ולהימנע משימוש בתווים בעברית בנתיב לתיקיות העבודה. זאת כדי למנוע בעיות תאימות אפשריות עם כלי הפיתוח והקומפיילרים שבהם נשתמש.
לדוגמה, ניתן ליצור תיקייה בשם Intro2cs ובתוכה תיקיות בשם ex0, ex1 וכן הלאה.

התקנת Git

כפי שהוסבר בתרגול Git הוא כלי לניהול גרסאות (Version Control System), המאפשר לעקוב אחר שינויים בקבצים לאורך תהליך הפיתוח של פרויקט.
באמצעות Git ניתן לתעד שינויים בקוד, לחזור לגרסאות קודמות ולנהל את התפתחות הפרויקט באופן מסודר.
אחד היתרונות המרכזיים של Git הוא האפשרות לשיתוף פעולה בין מספר מפתחים. כל מפתח יכול לעבוד על עותק משלו של הפרויקט, ובהמשך ניתן לשלב את השינויים שבוצעו בעותקים השונים.
חשוב להבדיל:

- Git הוא כלי המותקן במחשב ומאפשר לנהל גרסאות של קבצים ולעקוב אחר שינויים בהם.
- GitHub הוא שירות מקוון המאפשר לאחסן ולשתף פרויקטים המנוהלים באמצעות Git. פרויקטים אלה נקראים מאגרים (Repositories).

במסגרת הקורס נשתמש ב-Git בעיקר כדי להוריד למחשב האישי את קובצי התרגילים ממאגרים ב-GitHub, לעבוד עליהם באופן מקומי ולנהל את השינויים בקוד.

בשלב זה נתקין את Git במחשב ונבדוק שההתקנה הושלמה בהצלחה.

1. במחשבים שמריצים MacOS הרבה פעמים Git מותקן כחלק מחבילת ה-Command Line של אפל.
   פתחו את הטרמינל באמצעות לחיצה על תפריט Applications ומשם לחצו על Utilities או שתחפשו זאת באמצעות Spotlight
2. הריצו בטרמינל:

```
xcode-select --install
```

3. אם MacOS אומר שהכלים מותקנים אין צורך לעשות משהו נוסף.
4. בחלון שורת הפקודה, הקלידו את הפקודה הבאה ולחצו על Enter:

```
git --version
```

פקודה זו מציגה את גרסת Git המותקנת במחשב.
אם ההתקנה בוצעה בהצלחה, אמורה להופיע הודעה הדומה להודעה הבאה:

```
version 2.x.x (Apple git-xxx)
```

מספר הגרסה המדויק עשוי להשתנות בהתאם לגרסה שהתקנתם.
אם הופיע מספר גרסה, Git מותקן במחשב שלכם ומוכן לשימוש.

התקנת סביבת פיתוח

סביבת פיתוח היא תוכנה שעוזרת לנו לכתוב ולפתח קוד. היא יכולה להציג את הקוד בצבעים, להציע השלמות ולהקל על מציאת שגיאות.
ישנם מספר סביבות עבודה שמאפשרות גם התקנות של חבילות הרחבה שונות.
מוזמנים לבחור ולהתקין את הסביבה שנוחה לכם.

- Visual Studio Code
- Clion (שימו לב שישנה אפשרות לקבלת רישיון לסטודנטים כאשר מזינים מייל אוניברסיטאי בלינק הבא)
- sublime

התקנת הקומפיילר GCC

כשאנחנו כותבים תוכנית ב־C, אנחנו יוצרים קובץ טקסט שמכיל קוד מקור. כדי להריץ את התוכנית צריך לתרגם את הקוד לקובץ הרצה. את תהליך הבנייה נפעיל בעזרת GCC שהוא הקומפיילר הרשמי שנשתמש בו בקורס.
ב-macOS כשאנחנו מריצים GCC אנחנו בפועל את הקומפיילר clang של אפל, לכן גם אם תכתבו GCC כרגע אתם תקבלו שהקומפיילר קיים, פשוט הוא לא תואם לGNU GCC שאיתו אנחנו עובדים בקורס.
לכן נצטרך להתקין אותו באמצעות Homebrew.

1. פתחו את brew.sh והעתיקו את פקודת ההתקנה הרשמית שמופיעה שם שתתקין אצלכם Homebrew.
   הדביקו אותה בטרמינל ועקבו אחרי השלבים שם עד הסוף.
2. הריצו את הפקודות הבאות בטרמינל:

```
brew update
brew install gcc
ls $(brew --prefix)/bin/gcc-*
```

3. Homebrew נותן ל-GNU GCC שם שכולל את הגרסה בשביל לא להתבלבל עם הקומפיילר של אפל. לכן בכל זמן שנכתוב gcc-16 יש לשים לב שהגרסה יכולה להיות אולי חדשה יותר אצלכם לכן תחליפו את הגרסה במה שמתאים למחשב לכם (שמגיעה כפלט של הפקודה האחרונה בחלק העליון).

4. ניתן לבדוק שההתקנה הצליחה באמצעות הפקודה:

```
gcc-16 --version
```

כעת GCC מותקן במחשב ואפשר להתקדם הלאה.

היכרות קצרה עם שורת הפקודות

כפי שראינו בתרגול, אפשר לבקש מהמחשב לבצע פעולות שונות באמצעות פקודות טקסט מהטרמינל. אנחנו כעת נשתמש בטרמינל של mac. כדי לפתוח אותו, חפשו `Terminal` בspotlight.

לכל חלון כזה יש תיקיית עבודה נוכחית. פקודות שמציינות שם קובץ בלי נתיב מלא יחפשו אותו ביחס לתיקייה הזו. לכן, לפני שבונים תוכנית, צריך להיכנס לתיקייה שבה נמצא קוד המקור שלה.

שלוש פקודות שימושיות הן:

פקודה	משמעות
`mkdir`	יצירת תיקייה
`cd`	מעבר לתיקייה
`ls`	הצגת הקבצים והתיקיות בתיקייה הנוכחית

כהתנסות מוזמנים ליצור תיקייה לקורס ולעבור אליה באופן הבא:

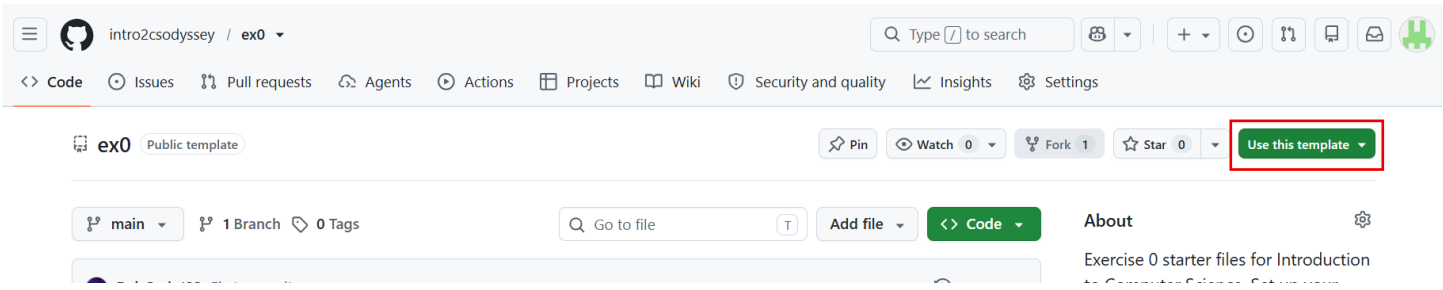
```
mkdir -p ~/intro2cs
cd ~/intro2cs
```

הפקודה הראשונה יוצרת את התיקייה. השנייה עוברת אליה. הסימן ~ מייצג את תיקיית הבית שלכם.
אם התיקייה כבר קיימת, אפשר להמשיך לפקודה השנייה.
את הפקודות שמופיעות במדריך מקלידים שורה אחת בכל פעם ולוחצים Enter אחרי כל שורה.

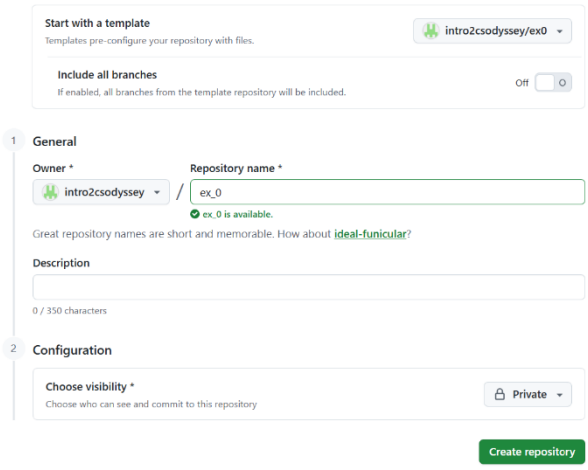
הורדת קובצי התרגיל באמצעות Git

פתחו את המאגר של תרגיל 0 ב-GitHub. המאגר מכיל את הקבצים שמהם מתחילים לעבוד.

נרצה ליצור מאגר אישי שנוכל לעבוד עליו באמצעות התבנית של התרגיל.
מה לגבי הכפתור Fork שראינו בתרגול? הכפתור יוצר עותק שמקושר למאגר המקורי ב-GitHub. אם מאגר המקור ציבורי, גם העותק שנוצר בלחיצה עליו ציבורי.
לכן אנחנו נעשה משהו שונה. אנחנו נשתמש בתבנית של התרגיל באמצעות הכפתור הבא:



ונבחר באופציה Create a new repository שתיצור לנו מאגר חדש שמבוסס על המאגר של התרגיל.
בעמוד שנפתח עליכם לקרוא למאגר החדש באיזה שם שתרצו (מומלץ ex0 כשם התרגיל המקורי) ועליכם לשנות את ההרשאה לפרטית:



וכעת אתם יכולים לראות שנוצר לכם מאגר מקורי משלכם!

כפי שראינו בתרגול, הפעולה clone יוצרת במחשב עותק מקומי של המאגר, כולל היסטוריית הגרסאות שלו. נבצע אותה פעם אחת עבור התרגיל.

1. פתחו בדפדפן את הקישור למאגר החדש שיצרתם בחשבון שלכם.
2. לחצו על הכפתור `Code`.
3. בחרו `HTTPS` והעתיקו את הכתובת שמופיעה.
4. ב-`Terminal`, מתוך תיקיית הקורס, הריצו את הפקודה הבאה:

```
git clone <repository-url> ex0
```

החליפו את `<repository-url>` בכתובת שהעתקתם, כולל הסרת הסוגריים הזוויתיים. המילה `ex0` שבסוף הפקודה קובעת את שם התיקייה שתיווצר במחשב שלכם.
כעת היכנסו לתיקייה ובדקו את תוכנה.
(לאמיצים ניתן לעשות זאת עם הטמרינל גם כאן באמצעות הפקודות שלמטה, כאשר מפעילים שורה אחר שורה).

```
cd ex0
ls
```

פתחו בדפדפן את הGithub שלכם ודאו שהמאגר נמצא בחשבון שלכם, מסומן Private ומכיל את קובצי התרגיל.

בניית התוכנית והרצתה במחשב

בקובץ המקור תראו תוכנית קצרה שמדפיסה הודעה למסך. תוכנית מסוג זה נראית כך:

```
#include <stdio.h>
int main(void)
{
printf("hello world\n");
```

```
return 0;
}
```

בדקו שהקובץ אכן קיים ונפתח ב-IDE שבו בחרתם (וודאו שאתם מבינים את הקוד כפי שראינו בתרגול ובהרצאה!).
חזרו ל-terminal וודאו שאתם בתיקיית התרגיל. כדי לבנות את התוכנית, הריצו:

```
gcc-16 -std=c99 -Wall -Wextra ex0.c -o ex0
```

לכל חלק בפקודה יש תפקיד:

- `gcc` – הפעלת הקומפיילר.
- `-std=c99` – בחירה בתקן השפה C99 (התקן הרשמי של הקורס).
- `-Wall -Wextra` – הפעלת אזהרות על בעיות אפשריות בקוד.
- `ex0.c` – קובץ המקור.
- `-o ex0` – קביעת שם קובץ ההרצה.

בבנייה מוצלחת של התוכנית הזו, בדרך כלל לא תופיע הודעה, ותחזרו לשורת הפקודה.
אפשר להריץ `ls` ולראות שנוסף הקובץ `ex0`.

כעת הריצו את התוכנית:

```
./ex0
```

עבור הקוד הנ"ל הפלט אמור להיות:

```
hello world
```

בנייה והרצה הן שתי פעולות נפרדות. לאחר כל שינוי בקוד צריך לשמור את הקובץ, לבנות שוב ולהריץ את קובץ ההרצה המעודכן.
בדקו הצלחתם לבנות את התוכנית ולהריץ אותה, והפלט מתאים לקובץ שסופק בתרגיל.

ננסה כעת שינוי מכוון כדי לראות כיצד המהדר עוזר לנו לזהות בעיה.

1. בקובץ המקור מחקו את התו ; שמופיע בסוף השורה עם `printf`.
2. שמרו את הקובץ ובנו שוב באמצעות GCC.
3. קראו את הודעת השגיאה הראשונה. חפשו בה את שם הקובץ ואת מספר השורה.
4. בדקו את השורה שצוינה ואת השורה שלפניה.
5. החזירו את התו שמחקתם, שמרו ובנו מחדש.

לאחר מכן החליפו את המילה hello במילה bye, שמרו, בנו והריצו. בדקו שהפלט השתנה.
בסיום החזירו את הקוד ואת הפלט הנדרשים בתרגיל, ובנו והריצו שוב.
אם ניסיון הבנייה נכשל, ייתכן שקובץ ההרצה מבנייה קודמת עדיין קיים. הרצה שלו תציג את ההתנהגות של הגרסה הקודמת. לכן יש לבדוק שהבנייה החדשה הסתיימה בהצלחה לפני שמריצים.

שמירת גרסה באמצעות Git

שמירה בעורך מעדכנת את הקובץ במחשב. כדי לתעד נקודת שמירה בהיסטוריה של Git ניצור `commit`.
לפני הקומיט הראשון הגדירו את השם ואת כתובת הדואר שיופיעו כמחברי השינוי. הריצו את הפקודות מתוך תיקיית התרגיל:

```
brew install gh
gh auth login
```

ובחרו ב-Login with a web browser. לאחר מכן בחרו ב-HTTPS ומשם המשיכו לעקוב אחרי ההנחיות על מנת לחבר את המשתמש שלכם.
כדי לתרגל שמירת גרסה בלי לשנות את הפלט, הוסיפו בתחילת קובץ המקור הערה:

```
/* My first C program */
```

הערה נועדה לקוראי הקוד. היא אינה מוסיפה הדפסה בזמן הרצת התוכנית.
שמרו, בנו והריצו כדי לבדוק שהתוכנית עדיין פועלת.
בחרו את קובץ המקור לקומיט הבא:

```
git add ex0.c
```

או כפי שהסברנו בתרגול השתמשו בנקודה כדי לבחור את כל הקבצים שבתיקייה:

```
git add .
```

כעת צרו את הקומיט, עם הודעה שמתארת את השינוי:

```
git commit -m "Add a comment to the first program"
```

עד עכשיו הקומיטים שיצרתם נשמרו במחשב שלכם. כדי לגשת אליהם גם ממחשב אחר נוכל לשלוח אותם למאגר פרטי בחשבון ה־GitHub שלכם.
נעלה באמצעות הפקודה:

```
git push
```

העלאת הקוד ל־GitHub שומרת עותק מרוחק של העבודה. בהמשך התרגיל נבצע גם את ההגשה במערכת הקורס.

היכרות עם שרת הקורס

שרת הוא מחשב מרוחק שאפשר להתחבר אליו דרך הרשת. שרת המחלקה מריץ Linux, ובו נוכל לבנות ולהריץ את הקוד בסביבת העבודה של הקורס.
עד עכשיו עבדנו במחשב האישי שלנו, שמריץ macOS. לאחר ההתחברות לשרת, הפקודות שנקליד בחלון המרוחק יתבצעו במחשב שמריץ Linux.
קוד המקור שלנו הוא קובץ טקסט שאפשר להעביר לשרת. שם נבנה ממנו קובץ הרצה חדש שמתאים לסביבה המרוחקת.

הכניסה לשרתי המחלקה דורשת גם קוד אימות מתחלף:

1. התקינו יישום אימות, כגון Microsoft Authenticator או Google Authenticator.
2. היכנסו לאתר הרישום לשירות האימות והזינו את שם המשתמש המחלקתי הקצר שלכם.
3. לאחר אימות שם המשתמש ישלח QR Code למייל האוניברסיטאי שלכם. פתחו את יישום האימות וסרקו את ה-QR Code.
4. בדקו שהיישום מציג קוד מתחלף (כל 30 שניות) עבור חשבון האימות המחלקתי. אתם תשתמשו בקוד זה על מנת להתחבר לשרתים המחלקתיים.

את הרישום מבצעים פעם אחת. בהתחברויות הבאות משתמשים בקוד העדכני שמופיע ביישום. להוראות האימות של הפקולטה במידה ומשהו לא מסתדר.

בשביל להתחבר לשרת המחלקתי פתחו את הטרמינל והריצו את הפקודה הבאה כאשר אתם מחליפים את `username` בשם המשתמש המחלקתי שלכם:

```
ssh -J username@vpn.esc.biu.ac.il:2222 username@planet.cs.biu.ac.il
```

התחברו והזינו את הסיסמה ואת קוד האימות לפי הבקשות שמופיעות.
ייתכן שתתבקשו להזין סיסמה יותר מפעם אחת. בזמן הקלדת סיסמה בטרמינל, לעיתים לא מוצגים תווים כלל.
הקלידו אותה ולחצו Enter.
להוראות ההתחברות של הפקולטה במידה ומשהו לא מסתדר.

אופציונלי: כדי לא להקליד את הפקודה הארוכה בכל פעם, אפשר ליצור (או לערוך) את הקובץ `~/.ssh/config` ולהוסיף בו:

```
Host planet
    HostName planet.cs.biu.ac.il
    User USERNAME
    ProxyJump USERNAME@vpn.esc.biu.ac.il:2222
```

מעתה אפשר להתחבר בפקודה `ssh planet`.

לאחר ההתחברות, הפקודות שתקלידו מתבצעות בשרת. צרו תיקייה לתרגיל ועברו אליה:

```
mkdir -p ~/intro2cs/ex0
cd ~/intro2cs/ex0
pwd
```

הסימון ~ מייצג את תיקיית הבית האישית שלכם בשרת. הפקודה `pwd` מציגה את נתיב תיקיית העבודה הנוכחית.
שימו לב - ייתכן שהקשה על הפקודות בשרת תיראה זהה לפקודות במחשב שלכם. כדי לדעת איפה אתם, הסתכלו על שורת הprompt, שבשרת תכלול את שם השרת.

פתחו חלון טרמינל נוסף במחשב שלכם (לא את זה שמחובר לשרת). עברו לתיקיית התרגיל המקומית והריצו:

```
scp -J USERNAME@vpn.esc.biu.ac.il:2222 ex0.c USERNAME@planet.cs.biu.ac.il:~/intro2cs/ex0/
```

או אם הגדרתם את הקיצור:

```
scp ex0.c planet:~/intro2cs/ex0/
```

תתבקשו שוב להזין סיסמה וקוד אימות. שימו לב ששם הקובץ והנתיב צריכים להיות בלי תווים בעברית.
כעת, בטרמינל של השרת, הציגו את רשימת הקבצים:

```
ls
```

בדקו שקובץ המקור מופיע, ואז בנו אותו:

```
gcc -std=c99 -Wall -Wextra ex0.c -o ex0
```

והריצו אותו:

```
./ex0
```

השוו את הפלט לפלט הנדרש בתרגיל. בדקו גם אותיות גדולות וקטנות, רווחים ומעברי שורה.
אם שיניתם את הקוד במחשב האישי אחרי שהעברתם אותו לשרת, העבירו שוב את הקובץ המעודכן ובנו אותו מחדש בשרת.

אם אתם רוצים להרגיש יותר בנוח עם Linux ועם פקודות בטרמינל (כמו הפקודות שהשתמשנו בתרגיל) אנחנו ממליצים לקרוא את המדריך הבא. אתם לא חייבים לקרוא אותו והרבה מהפקודות שבו לא ישמשו אותנו במהלך הקורס, אך מי שרוצה להעשיר את הידע מוזמן בשמחה.

הגשת התרגיל

בכל תרגיל יש לפעול לפי שמות הקבצים ופורמט ההגשה שפורסמו עבורו. מערכת בדיקה אוטומטית עשויה להסתמך גם על שם הקובץ ועל מיקומו בתיקייה.

באופן כללי, אם נדרשת הגשה של קובץ מקור בודד, בחרו את קובץ ה-C המעודכן. אם נדרש ZIP, צרו אותו מהקבצים שנדרשו ופתחו אותו לפני ההעלאה כדי לבדוק את תוכנו. כללו את תיקיית `.git`.

שימו לב – באופן דיפולטיבי לא תמיד תראו את התיקייה `.git`. מכיוון שהיא תיקייה מוסתרת. ב-mac בfinder אפשר להשתמש ב-Cmd+Shift+. (נקודה).
את קובצי ההרצה שנוצרו במהלך העבודה, כגון `ex0`, אפשר ליצור מחדש מקוד המקור ולכן אין צורך להגישם.

בשביל ליצור קובץ ZIP (מעניין תיקייה דחוסה של הקבצים בשביל להקטין את גודלם) פתחו את תיקיית ההגשה שעבדנו איתה ב-Finder ובחרו את הקבצים הנדרשים. לחצו באמצעות קליק ימני עליהם ובחרו Compress Items.
ה-Finder יוצר תיקיית ZIP באותה תיקייה. שנו את השם ל-ex0.zip.

בהגשה של תרגיל זה עליכם להגיש בסופו של דבר תיקיית ZIP בשם ex0 שבתוכה יש את תייקת ה-`.git`. וex0.c בלבד!

כעת צריך להגיש במערכת ההגשה:

1. פתחו את מערכת ההגשה.
2. התחברו באמצעות פרטי המשתמש והאימות הנדרשים.
3. בחרו את הקורס (891100), הקבוצה (01) ותרגיל ex0.
4. בחרו את הקובץ להגשה והשלימו את אישור ההגשה.
5. פתחו את המשוב שהתקבל בדואר האוניברסיטאי.



בתרגיל זה, משוב עם ציון 100 מאשר שהגשת הניסיון עברה בהצלחה. אם התקבל משוב אחר, קראו את ההודעה, תקנו את הבעיה, בדקו שוב והגישו את הגרסה המעודכנת. גם לאחר שהתקבל ציון 100, ודאו שהשלמתם את הבנייה וההרצה המקומיות ואת הבדיקה בשרת.

פתרון תקלות נפוצות

- הפקודה `git` או `gcc` אינה מזוהה - סגרו את הטרמינל ופתחו חדש. אם הבעיה נמשכת, חזרו על שלב ההתקנה. ב-Mac וודאו שחלון ההתקנה של `xcode-select` הסתיים.

- Permission denied בהרצה - ודאו שהקובץ נוצר בהצלחה בשלב הבנייה. ייתכן שהבנייה נכשלה.

- שיניתי את הקוד אבל הפלט נשאר זהה - בדקו ששמרתם את הקובץ, שהבנייה הצליחה ושהרצתם את קובץ ההרצה מהתיקייה הנכונה. אם אתם עובדים בשרת, ודאו שהעברתם אליו את גרסת המקור המעודכנת ובניתם אותה שם.

- Git מציג nothing to commit - ההודעה אומרת שאין כרגע שינויים שנבחרו לשמירה בקומיט חדש. בדקו ששמרתם את הקובץ בעורך, שאתם בתיקיית המאגר הנכונה ושהרצתם `git add` על הקובץ ששיניתם.

- ההעלאה ל-Github נכשלת - בדקו את כתובת היעד באמצעות הפקודה:

```
git remote -v
```

ודאו שהיעד מצביע למאגר האישי שלכם. השלימו את תהליך ההתחברות אם נפתח דפדפן. אם ההודעה עדיין אינה ברורה, שמרו אותה ופנו לפורום הקורס.

- ההתחברות לשרת נכשלת - בדקו את שם המשתמש, כתובת השרת, פרטי השער והפורט. ודאו שאתם משתמשים בקוד האימות העדכני של החשבון המחלקתי.
כדי לקבל עזרה, צרפו את הודעת השגיאה ואת השלב שבו הופיעה. שמרו את הסיסמה ואת קוד האימות לעצמכם.

לפני שמסיימים

עברו על הרשימה ובדקו שכל אחד מהשלבים הבאים הצליח:

- ☐ פתחתי את תיקיית התרגיל בעורך, בניתי את התוכנית במחשב שלי והרצתי אותה. ראיתי את הפלט המבוקש.
- ☐ יצרתי בכוונה שגיאה בקוד, קראתי את ההודעה ותיקנתי את הקוד. בסיום בניתי והרצתי שוב עם הפלט הנדרש.
- ☐ שמרתי שינוי ב־Git. ואני רואה אותו גם במאגר האישי שמסומן Private ב־GitHub.
- ☐ העברתי את קובץ המקור המעודכן לשרת המחלקה, בניתי אותו עם GCC והרצתי אותו שם.
- ☐ פתחתי את `ex0.zip` ובדקתי שקובץ המקור `ex0.c` ותיקיית `.git` מופיעים ישירות בתוך הארכיון.
- ☐ אישרתי את ההגשה במערכת הסבמיט וקיבלתי בדואר האוניברסיטאי משוב עם ציון 100.

אם אחד השלבים עדיין חסר, חזרו לסעיף המתאים. אם נתקעתם, ציינו בפנייה לפורום את מערכת ההפעלה שלכם, את הפקודה שהרצתם ואת הודעת השגיאה שקיבלתם.